- *Opening*
  - Hello, everyone.
  - I'm Kattni and this is Switching from Sphinx to Markdown.
  - Today I'm going to talk about reStructuredText and Markdown, the associated documentation options, and what goes into migrating a substantial existing set of documentation from the first to the second.
  - This project involved several tools, which are mentioned throughout this talk, and you can find a full list of resources linked at the end of the talk.
  - You may be aware of the MkDocs documentation system due to the shift currently happening in the ecosystem.
  - I will be expanding on that towards the end.
  - The thing to know heading into this is that everything I discuss today is still relevant and applicable.
  - First, a bit about me.

    - The important thing you need to know is that I am tolerated...
- *Kittens*
    - ...by a cat and three kittens who continue to let me live with them.
    - Beyond that, in my work life, I am the Superpowers Community Engineer with Prime Radiant, where I am working to engage with our users and build a sustainable community.
    - In another part of my life, I'm a core developer on the BeeWare project, where my primary focus is documentation, and I acted as technical project manager for the BeeWare documentation infrastructure migration from reStructuredText and Sphinx, to MkDocs and Markdown.
- *Acknowledgement of country*
    - Before I start, you may have noticed from my accent that I'm not from around here.
    - I'd like to acknowledge that I am from the lands of the Bodéwadmi and Odaawaa peoples - otherwise known as southeast Michigan, US - and I'd like to recognize that we are meeting

today on the lands of the Turrbal people, to recognise their continuing connection to their lands, waters, and culture, and to pay my respects to their elders past, present, and emerging.

- *reStructuredText (reST) and Sphinx*
  - *What is it?*
    - reStructuredText or reST, not "Rust", but "reST", is a markup language primarily used for documentation in the Python ecosystem, though it has seen adoption elsewhere as well.
  - *History*
    - Python needed a better way to do documentation than the existing LaTeX-based solution.
    - reST was designed with Python's docs in mind.
  - *Documentation generation*
    - Sphinx is a documentation generator developed for and used by the Python ecosystem, though it has also seen adoption by various other projects.

        - reST gives you a format for a single file, and provides a way to describe a block of text as, for example, bold, italics, or a paragraph.
        - Sphinx lets you create a set of related documents, that are interlinked, with additional tooling and plugins for specific tasks like API documentation.
    - *Ecosystem inertia*
        - One of the main reasons that reST is still used in the Python ecosystem is because it is a capable tool with an extensive history as the Python documentation format.
        - reST has been in use for Python for 23 years, and Sphinx for 18 years.
        - CPython 2.6 replaced LaTeX with Sphinx.
        - Other projects, such as Django and BeeWare, have continued it's use because it was used by everything that came before it.
    - There are, however, alternatives.
- *Markdown*

- *What is it?*
    - Markdown is also a markup language for creating formatted text.
- *History*
    - Markdown was designed to be a human-readable, human-writeable solution to document formatting that can be easily converted to HTML.
    - It was originally written to simplify blogging.
    - The web is based on HTML, as a markup language, but HTML is verbose and unwieldy for a human to write.
    - If you're writing a blog, you need to be able to produce HTML quickly.
    - This is where Markdown came in.
    - However, it's become much more widely used.
- *Adoption*
    - Markdown has been adopted across many tech ecosystems.
    - Nearly all major messaging platforms, including apps like Slack and Discord, support using it to format messages.

    - Many text and code editors not only support it, they include a rendering preview feature available.
    - GitHub chose Markdown to be the default format, going so far as to develop their own "flavor" - GitHub-Flavored Markdown.
- *AI integration*
    - AI is all built around processing human readable text.
    - The simplicity of Markdown being able to move to HTML from text has worked to its advantage with the AI text processing pipeline.
    - If you look at the UI for LLMs such as Claude or ChatGPT, they all accept Markdown formatting.
- *Documentation options*
    - Markdown is now a viable alternative to reST for documentation, due to the development of various tools.
    - There are several options including platforms such as MkDocs, Docsify,

Docusaurus, GitBook, and teedoc, as well as Sphinx when using the MyST parser.
        - Essentially, MkDocs, etc, is to Markdown, as Sphinx is to reST.
    - How do the two formats compare?
- *Pros and cons of reST*
    - *What it does well*
        - In terms of what it does well, reST provides rich cross-referencing of multiple documents of content.
        - It was for a long time, the only option for using Sphinx, the combination of which was originally the only backend supported by Read the Docs.
    - *Difficulties it presents*
        - As for difficulties it presents, reST was not concerned with being human readable.
        - Link syntax is obtuse at best.
        - It's very whitespace-sensitive, and doesn't allow for markup in the link text, including formatting text as code.
    - *(reST example)*

        - This is an example of a simple document using reST formatting.
        - It contains a document heading, followed by a paragraph that includes bold text, strikethrough text and a web link.
        - The strike role and class must be included at the top to render strikethrough text, however, even that doesn't work on its own, it requires a separate CSS element to successfully render.
        - The number equals signs around the heading must always match the length as the heading text, so you have to remember to update that if you alter the text.
        - To render properly, the link must include the backticks, the space between the text and the URL, the brackets, and the final underscore.
- *Pros and cons of Markdown*
    - *What it does well*
        - As for what Markdown does well, it was designed to be human readable and easily converted to HTML.

    - Reference links are written in a way that flows naturally, and allows markup in the link text.
    - The rest of the syntax is straightforward and can typically be picked up quickly.
    - It is much closer, practically speaking, to how you would write a text document using ASCII-based emphasis.
- *Difficulties it presents*
    - As for difficulties it presents, there is generally no agreement on what the canonical flavor of Markdown is.
    - The creator, and much of the community, has actively worked against the idea.
    - As a result, there are several competing flavors.
    - The closest to a standard is CommonMark, but it has not been fully adopted.
    - In terms of use for documentation, MkDocs and other tools were not developed until relatively recently compared to Sphinx.
- *(Markdown example)*

        - This is the same document, formatted in Markdown.
        - Heading levels are determined by the number of hashes at the beginning of the line.
        - Link text is included in square brackets, immediately followed by the link target in parentheses.
    - So...
- *Which one is better?*
    - Which one is better?
    - Both reST and Markdown clearly have strengths and weaknesses.
    - As with many things...
    - (click)
    - It depends on your situation.
    - That said, the overwhelming adoption of Markdown by a broad range of ecosystems speaks to its capabilities.
    - I would argue that if ease of maintenance and approachability are important to your project, Markdown is a better choice.

        - I'm not suggesting that Sphinx and reST are bad.
        - I'm am, however, saying that it can become easier to make a decision to make a major shift when you are adapting to the flow of overall change.
- *Migrating BeeWare: Origin story*
    - *How it began*
        - The BeeWare documentation migration process really began two years ago, though the bulk of it is more recent.
        - For a little context, BeeWare is a suite of tools designed to enable you to write cross-platform GUI applications entirely in Python.
        - You probably know the project lead, Russell Keith-Magee, sitting here in the front row.
    - *Becoming a contributor*
        - I decided to start contributing at the sprints at PyCon US 2024.
        - I struggled to find the documentation while preparing for my first call with Russell.

        - This issue became the topic of the call, and, subsequently, my primary contribution focus.
        - Fast forward to the lead up to PyCon US 2025.
        - We were dealing with issues related to reST, and Russell remarked:
    - (click)
        - "Once upon a time, reST seemed like a good idea, but I'm increasingly thinking moving to Markdown across the board would be worth it."
        - When I followed up with full support, he quipped:
    - (click)
        - "If only we had a sprint coming up..."
        - Little did he know.
        - That said, moving to Markdown across the board was not going to be a simple task.
- *Migrating BeeWare: The full scope*
    - *(no bullet)*

    - BeeWare is not a single library, it is multiple tools working under an umbrella.
    - The BeeWare project has a website, and each of the tools have their own documentation.
    - We use Read the Docs and GitHub pages for web deployment.
- *Four categories*
    - The BeeWare documentation infrastructure falls into four general categories:
- *Tutorial*
    - the Tutorial...
- *Major tools: Toga, Briefcase, Rubicon Obj-C*
    - the three major tools, Toga, Briefcase, and Rubicon Objective-C...
- *Standalone documents*
    - various standalone documents...
- *Website*
    - and the website.
    - This documentation structure may not necessarily be representative of your

situation, as many projects have only one set of documentation.
    - I'm going to share my experience shifting the BeeWare documentation infrastructure from reST and Sphinx to Markdown and MkDocs.
    - I began with the website content.
- *Website content* (click twice) (7:30)
    - This conversion took a few hours at the PyCon US 2025 sprints.
    - The BeeWare website was originally built using a system called Lektor.
    - *Lektor in reST mode*
        - Lektor is website builder that is halfway between a content management system and a static site generator.
        - It defaults to Markdown for content.
        - When the BeeWare website was created, Russell went to great lengths to get it into reStructuredText mode, to align with the rest of the documentation, which as mentioned earlier, seemed like a good idea at the time.

        - This was now a frustrating feature, and that made it a good first target for testing the viability of migration.
    - *Pandoc and Lektor API*
        - The tools used for this were a markup format converter called Pandoc, and a couple of features of the Lektor API.
        - The reST content was extracted using the API, and was converted using Pandoc.
    - *Proof of reST to Markdown*
        - The most important thing to come out of this was proof that it was possible to convert reST to Markdown with some level of automation.
        - This opened the door for migrating the entire documentation infrastructure.
- *Preparation for project migration* (click twice)
    - Preparation for the full project migration took about one and a half months.
    - When I began this process, the switch to Markdown was not a guarantee.
    - My job in this was to explore the potential and prove it was possible.

- I went into this with the understanding that, if I couldn't prove viability, the project would be scrapped.
- There were several pieces to the process, the details of which I will get into in the next sections.
- This included:
- (click)
- considering our backend options
- (click)
- choosing and investigating MkDocs
- (click)
- creating a custom translation solution
- (click)
- exploring the add-ons we would need to shape our docs
- (click)
- customising and writing various migration tools
- (click)
- choosing what tools we needed to replace the various checks we were doing, and putting together a repo for shared content and tooling.

- *Preparation: Consider options*
  - *Requirements*
    - The initial discussion involved a list of requirements and ideal features.
    - Requirements included that the documentation be a static site, that the platform support translations, and that the overall setup have, at the very least, basic feature parity with Sphinx.
    - High on the list of ideal features was direct documentation of type aliases.
    - With that in mind, I considered our options.
  - *Sphinx and MyST parser*
    - Sphinx does actually have content support for Markdown.
    - However, the API documentation generation still requires reST, so you're then still dealing with reST, on top of now dealing with inconsistency in your documentation formatting.
  - *MkDocs*
    - MkDocs is a static site generator built around Markdown, and aimed at

documentation.
        - It definitely had the potential to fit our needs.
    - *Other alternatives*
        - As you look at other projects that are actively being developed, MkDocs comes out as most common documentation solution.
        - This includes projects such as FastAPI, Textual and rich, cibuildwheel, and Pydantic.
        - I was offered the opportunity to look into alternatives.
        - I chose to dig into MkDocs first.
- *Preparation: Investigate MkDocs*
    - *Learn the basics*
        - The first step was to learn the basics of working with MkDocs.
    - *Research features*
        - Once I had that down, I needed to establish basic feature parity with Sphinx.
        - Verifying that the most basic features of Sphinx were available was a quick

process; establishing full feature parity would take much longer.

- I chose early on to figure out documenting type aliases.
- Russell mentioned offhand that if this was doable, it would mean an instant approval of the change.
- This is because there are some features of the current Python typing experience that are not well supported by sphinx-autodoc.
- While I knew this wouldn't actually guarantee approval, I'm always up for a challenge, and while it took some doing, I was able to get it working.
- One of the things we discovered early in the process, was that the live preview build was quicker and more responsive than Sphinx had been.
- As well, unlike Sphinx, MkDocs was capable of tracking changes to the documentation in the source code in the live build, without any decrease in performance.

- *Material theme*
    - Another important piece of this was choosing a theme.
    - There were several available, however, Material for MkDocs had the most useful features, and appeared to be the most actively maintained.
- *Establish viability*
    - I was confident at this point that a majority of what we needed was feasible with MkDocs.
    - With basic viability established, I turned to translations.
- *Preparation: Build and translations*
    - *No viable translation options*
        - MkDocs had a "multilingual" story, but no real translation story.
        - The MkDocs recommendation involved having a local, static translated copy of your entire site in Markdown for every language you required.
        - This wasn't going to work for us.

    - We were using an online, community-driven translation front-end called Weblate.
    - In order to make efficient use of Weblate, you need to present content in a translatable form, which means using Gnu Gettext PO files.
- *Translate Toolkit*
    - I found the Translate Toolkit, which is a set of translation tools, provided by the Weblate maintainer.
    - They allow for converting between, among other things, PO files and Markdown.
- *Build tooling*
    - Integrating the translation process into the build tooling would require writing wrappers around the translation and build commands.
    - We used tox, a frontend for executing project tasks in a clean environment, for running the wrapper scripts.
    - This meant, among other things, that when we deployed the documentation to Read the Docs and GitHub Pages, that what we

were running locally directly mirrored what was being run remotely.
        - This was incredibly beneficial in troubleshooting build issues and failures.
    - *Build integration*
        - I was able to build a process that used the translated content of the PO files to generate the associated Markdown in a temp directory, ran the MkDocs build process within, and generated translated HTML.
        - This solution would evolve as the migration progressed, but we were able to build on the basic concept I provided initially.
- *Preparation: Explore add-ons*
    - *(no bullet)*
        - To reproduce our existing documentation, I would need to employ a number of extensions and plugins.
        - These are the ones common to all of our projects.
    - `attr_list`

    - The Attribute List extension added syntax to define attributes on the HTML output.
    - This allowed me to apply CSS to any element by adding an ID within the Markdown.
- `mkdocs-autorefs`
    - Autorefs added syntax for simple linking between pages.
- `mkdocs-macros`
    - Macros is a plugin and mini-framework meant to add macro and automation capabilities through the use of custom variables, macros, and filters.
    - We made significant use of this add-on throughout the project.
    - It enabled us to do things like customise the shared content experience through configuration variables, and automate content creation based on metadata.
- `mkdocs-literate-nav`
    - The basic navigation built into MkDocs wasn't sufficient for our needs.

    - Literate Nav allows for specifying custom page navigation structures in Markdown, instead of within the YAML configuration file.
- `mkdocstrings`
    - For those familiar with Sphinx, mkdocstrings is the analogue to sphinx-autodoc.
    - For those unfamiliar, this is the plugin designed to extract documentation from docstrings in your codebase.
    - It has extensive configuration options, including things like overall formatting, and how and what class and typing information is displayed.
- *PyMDown Extensions*
    - PyMdown Extensions was critical.
    - Called as pymdownx, it supplies several Markdown formatting options, including code syntax highlighting, admonitions, tabbed content, and external content injection.

        - The next step was automating the Markdown conversion.
- *Preparation: Customise Pandoc*
    - *(no bullet)*
        - MkDocs makes use of quite a number of plugins and extensions that apply to Markdown formatting.
        - Pandoc is, by nature, unaware of the existence of these add-ons, and therefore doesn't know how to handle converting to exactly what was needed for our documentation.
        - It took a lot of trial and error to figure out getting as close as possible.
    - *Pandoc Markdown extensions*
        - Pandoc offers several Markdown extensions that handle converting various elements of reST.
    - *CommonMarkX*
        - CommonMarkX is Pandoc's implementation of CommonMark, which includes a specific set of the Pandoc

extensions, to cover features that aren't strictly part of CommonMark.

- *Tailoring the Pandoc command*
    - Pandoc is designed to be run on one file at a time.
    - I used `find` combined with Pandoc to enable running the command over all files in a directory.
    - This...
    - ...is the command I used.
    - As shown, it is possible to customise which extensions are included with CommonMarkX when running the Pandoc command.

- *Preparation: Conversion scripts*
    - *Content*
        - I used Pandoc for the initial conversion, but that left quite a bit of work to do.
        - I built a custom script of regular expressions to handle getting the Pandoc-converted content to a MkDocs-compatible state.
        - This...

        - (click)
        - ...is one of the most obnoxious.
    - *Docstrings*
        - I couldn't run Pandoc on the source code, so that meant converting docstrings entirely with regular expressions.
        - Luckily, I found that there was a limited set of reST markup elements used in the docstrings.
        - This task required special care to ensure that none of the source code was altered.
        - Version control was invaluable here.
    - *Translations*
        - The translations were also in a format that Pandoc wouldn't work with, so they also required a separate set of regular expressions.
        - The biggest issue I ran into here was that there were many malformed reST links.
- *Preparation: Linting and checks* (15:00)
    - *(no bullet)*

    - I wanted to be able to verify and maintain consistency in our Markdown content.
- *rumdl*
    - I was introduced early on to a markdown linter called rumdl, which was very early in its development.
    - It had a number of bugs, but also a phenomenal maintainer.
    - They opted to include a MkDocs setting that would encompass all the necessary features to work with MkDocs-compatible Markdown, which we've worked together on improving throughout this project.
- *PySpelling*
    - I chose PySpelling for our spell-checker.
    - It was more opinionated than our existing spell-checker, and required a longer custom word list, but once we sorted that, it's worked smoothly.
- *markdown-checker*
    - While it has other features, I'm using markdown-checker specifically to verify that web URLs are valid.

- *Preparation: BeeWare Docs Tools*
    - *(No bullet)*
        - As part of preparation, one of the things we did was build a separate utilities project called BeeWare Docs Tools.
    - *Sandbox for feature testing*
        - We built a basic website that exercises many of the features used in the rest of the documentation.
        - This provides us with a testbed where we can test the impact of changes ahead of deployment.
    - *Single source of dependencies*
        - We use this as a single source of dependencies for all the documentation.
    - *Theme unification*
        - We made a series of customisations to the theme, and this allows us to keep that in one place, and apply it across the board.
    - *Shared content*
        - The available tooling for injecting content from a separate location into the documentation meant we could

consolidate things like our contribution guide and style guides into one place, and have the same documentation across all projects.

- *Build tooling*
    - All of the build tooling, designed mostly around the translation solution, is maintained here.
    - Quite recently, we were chasing down a bug in a tool called click, and in the process, found a workaround for it.
    - The workaround involved adding a flag to the live-serve tooling.
    - If we weren't hosting all of the build tooling within one repo, we would have had to deploy this change across several.
    - This is not the first example of this utilities project proving to have been a good choice.
- That covers what went into preparing for the overall migration.
- At that point, I was given the go-ahead to migrate the tutorial.

- *Tutorial* (click twice)
    - Once preparation was done, the actual tutorial migration took two weeks.
    - *Good first project*
        - The tutorial was a good first choice because it was, relative to the rest of the documentation, simple.
    - *Relevant basic features*
        - While the documentation features of the tutorial are basic, they are all relevant to the rest of the project.
        - For example, it consists of eight sections that have interlinking between pages, and includes code-blocks and tabbed content.
    - *Translated content*
        - The tutorial was already fully translated, primarily by human translators, and we did our best to maintain as much of that content as possible.
- *Toga* (click twice)
    - The Toga migration took the better part of two and a half months.
    - *Most complex project*

    - Toga's documentation is the most complex and expansive documentation within BeeWare.
    - It has a large API, and also includes tutorials, reference guides, how-to documents, topic guides.
- *Proof of concept*
    - It required its own proof of concept.
    - Early in the overall process, I created a draft PR showing the full conversion three of the most complicated pages within the API documentation.
    - These pages contained examples of every element necessary for the rest of the documentation.
    - Most of the work that went into this part was manual, as I needed it to be as complete as possible, and I was still working on building the conversion script.
    - It took about three weeks before I was given the go-ahead to officially begin working on it.
- *Configuration*

    - As I said, Toga has a significant amount of API documentation.
    - This would be the first use of mkdocstrings.
    - This was one example of a vast improvement over the analogue Sphinx feature; mkdocstrings resulted in API documentation formatted differently than the existing, but we ultimately found it to be better.
- *Additional changes*
    - We took the opportunity to do some additional housekeeping when a literal port of the old code would be more difficult than a new option that would give us better outcomes anyway.
    - For example, we had tables being built from CSV files, and they are now being handled by a YAML parser.
- *Manual verification*
    - One of the concerns heading into this was that there would be critical things missed

during the migration process, that wouldn't necessarily be caught on review.
    - This was a major change, involving a lot of files, and the only way we could guarantee everything was handled was with manual verifications.
    - To ensure this, I went through the entirety of the newly rendered documentation site and compared it to the existing documentation site, page by page.
    - As expected, this caught a number of issues, and a few edge cases.
    - The complexity of Toga's documentation meant this was a required step.
    - With simpler documentation, it may not be a requirement, however, doing so across a simpler setup would take significantly less time, and, especially if this is your first migration, I highly recommend it.
- *Briefcase* (click twice)
    - Briefcase, on the other hand, took three days to migrate.
    - This was due to multiple factors.

- *Less complicated*
    - First, the documentation is significantly less complicated.
    - There is less overall documentation and no API because it's a user-facing tool, not a library.
- *Utilised existing tooling*
    - Second, the hard work had already been done.
    - Everything we had built up to this point was more than sufficient to handle the Briefcase migration, and it just worked.
    - We did a similar manual audit at the end, and caught very little in terms of issues.
- *Rubicon Objective-C* (click twice)
    - Rubicon Objective-C took about two weeks.
    - This increased time was because it presented new problems to solve.
    - *Typing file to the rescue*
        - While the documentation wasn't much more extensive than Briefcase, and Rubicon *is* Python, the code is written more like Objective-C.

        - This oddness means treating the source code as documentation turned didn't work out as well.
        - We managed to work around it by using a typing file, which preserves the way the code is formatted and runs, but allows for presenting the documentation as we wanted it.
    - *Unusual complications*
        - Unusual code leads to unusual complications.
        - The problems that we hit porting Rubicon are not representative of most of the problems that most projects will hit.
        - Most projects are going to be more like Toga or Briefcase, than Rubicon, and projects that hit complications like we did with Rubicon are going to hit their own unique complications because they're also doing something weird with the Python language.
- *Website* (click twice) (21:00)

- That left the website, the migration of which would take over three months.
- *Stretched the capabilities*
    - MkDocs is a documentation building tool.
    - A website is ostensibly documentation, but with a more promotional focus.
    - We wanted to maintain the general layout and content of the existing site.
    - This meant essentially building it from scratch in Markdown, which is what ended up making this project take the longest of all.
    - It stretched the capabilities of MkDocs, but ultimately, we were able successfully build the site.
- *Simplified content creation*
    - Content creation with Lektor had its issues.
    - We added some utilities to simplify content creation, which have significantly improved the experience for the team.
- *Unified tooling*
    - Once this was landed, BeeWare was using MkDocs across all projects.

        - This meant the look and feel of the BeeWare website and project documentation was unified, which vastly improved the user experience.
- *Standalone documents*
    - *(no bullet)*
        - The BeeWare project also had a series of standalone documents that were in reST.
        - We addressed these throughout the migration process.
    - *Change note system*
        - We automatically generate release notes from change notes included with every PR submitted to specific projects.
        - Migrating this process involved converting all the existing documents to Markdown, and updating the CI configuration to match.
        - The most difficult thing to overcome here is the muscle memory of some of the team, and reST change notes still show up from time to time.
    - *Templates*

        - Briefcase as a project has a number of child repositories that contain reST, that are templates that produce READMEs, which required updating.
    - *READMEs*
        - Every project also had a README in the project root that was reST.
        - These were updatable using a simple Pandoc conversion, and were very straightforward to verify.
- *The human element* (21:30)
    - *(No bullet)*
        - It's easy to look at this as a purely technical change of markup language from A to B.
        - But that's not the whole story.
        - It's also a social change, because with any project involving a team, there is a human element to contend with.
        - At the time that I started, I wasn't a member of the core team, and I was, in fact, a relatively new contributor.

    - I had spent a number of years as a technical writer, but had not worked on anything that I would consider substantive code.
    - I wanted to shift BeeWare to Markdown across the board.
    - The team wasn't so convinced.
    - We had something that worked.
    - Why change?
- *Presenting a successful proposal*
    - This meant putting together a proposal solid enough to convince even the most recalcitrant member of the core team.
    - I had to be prepared to address a continuing series of questions and concerns.
    - This was not a single step process, rather, it was an ongoing discussion throughout.
    - Presenting a successful proposal meant being able to show the project was viable, and that the results would be, at the very least, the same, if not better.
- *Deliverable, not just doable*

- Proving it was viable meant that it was deliverable, which goes beyond "can it be done" to "can it actually be delivered".
- Something being doable simply means that an end-state exists.
- Deliverable means there is a path to successfully get to the end-state within the given parameters.
- In this case, that meant being able to land this without a single, behemoth pull request.
- Don't get me wrong, there were several monster PRs necessary throughout the process.
- But overall, each step was kept as manageable as possible, and sufficiently self-contained that if I needed to step away for whatever reason, the project wouldn't fall apart.
- I was the primary developer on the migration, and there is always a risk in open source that volunteers become

otherwise occupied, and projects are left unfinished.

- *Stages*
    - To that end, I proposed the process in stages.
    - Each stage built on the previous.
    - There were multiple changes to multiple projects, which could be done piece by piece.
    - We could opt out of any given piece at any time without negating all the previous changes.
    - This enabled me to present this as a series of incremental changes, instead of an all-or-nothing situation.
    - In this case, incremental completion meant the inherent risk of me stepping away was significantly diminished.
    - The team wasn't the only human element.
- *Engaging the community*
    - When it came to our READMEs, we took the time to create labeled "good first

issue"s on every repository, with details on how to convert them.
    - It took a bit of work up front to create the issues, but it gave us an opportunity to engage with the community.
    - We were successful; almost all of this content was converted by members of our community.
- *Total time: Nine+ months*
    - So...
    - How long did it take?
    - The primary documentation infrastructure migration, from inception to completion, took...
    - (click)
    - ...over nine months.
    - To be clear, our process involved one developer.
    - That is an implementation detail, not a requirement.
    - This process could easily be split up amongst multiple people, and performed in parallel, significantly reducing the total time needed to complete it.

    - What does over nine months of work look like in numbers?
    - (click)
    - 37 PRs.
    - Nearly 3000 files modified.
    - Nearly four hundred thousand lines added, and just over three hundred thousand lines deleted.
    - That's a total of nearly seven hundred thousand lines touched.
    - The overall net was over 83 thousand added.
    - This is probably a bit low, but at least gives an idea.
- *Was it worth it?*
    - Was it worth it?
    - (click)
    - Absolutely.
    - We're in a much better place.
    - Incremental builds are lightning fast.
    - Types are natively documented.
    - The increased configurability of the API documentation was an immense improvement,

including being easier to read, better laid out, and more customisable.
    - Everything is in Markdown, which means the syntax is easier to consume.
    - The shared content avoids previous duplication of content.
    - The simplification of content creation means we are far more likely to write updates.
    - And, the documentation is now easier to maintain.
- *Would we do it again?*
    - Don't underestimate the amount of time and effort that went into this.
    - It took a lot of one developer's time, and a non-trivial amount of review and feedback from other developers on the core team, to land it.
    - So, would we do it again?
    - (click)
    - Unequivocally, yes.
    - It is a set of changes that wouldn't have happened if I hadn't come along to do it, but they are absolutely worthwhile, and have

resulted in a vastly better developer experience for everyone involved.
    - This leads to more easily crafted user experience improvements as well.
- *Should you do it?* (25:00)
    - Should you do it?
    - (click)
    - That's for you to decide.
    - Markdown is fast becoming the chosen format in many ecosystems, and is often more approachable than reStructuredText.
    - While much of the Python ecosystem is still using reST and Sphinx, Markdown is proving itself to be a viable alternative.
- *The future of MkDocs*
    - I want to note that MkDocs is currently in a transition period.
    - *MkDocs 1.x*...
        - MkDocs 1.x is no longer being maintained.
        - There are other options available.
    - *MkDocs 2.0...*

    - There is a MkDocs 2.0 which is not backwards compatible with MkDocs 1.
- *ProperDocs...*
    - ProperDocs is essentially a fork, aimed at preserving MkDocs 1.x.
- *Zensical...*
    - Zensical is a ground-up rebuild, designed to be a drop-in replacement, to the point of recognising the MkDocs configuration files as they already exist.
- *General migration process is the same*
    - The path to switching from Sphinx to ProperDocs or Zensical would closely mirror what I've talked about today.
- We're looking most seriously at Zensical for BeeWare.
- All of the work discussed today would be necessary regardless of the solution you choose.
- Hopefully I have given you an idea of the size and scope of what this change might look like for you, and ways to approach it that will make it achievable.

- *Find me:* (26:15)
    - Here's where you can find me on the internet.
    - The link at the bottom goes to my slides, and a resource list of links to all of the tools and projects mentioned in this talk.
    - Thank you.